Progetto Laboratorio di Ingegneria Informatica

Amatucci Michele **Cioce Valerio** **Ciurariu Beatrice**
2152202 2130244 2107235

1 Obiettivo del Progetto

Il progetto si occupa dello sviluppo di una pipeline end-to-end containerizzata in una struttura Docker per l'acquisizione, l'estrazione del testo e la valutazione qualitativa di documenti provenienti da domini web assegnati, tali: Wikipedia, Fifa, ESPN e Yahoo Finance. Il programma viene successivamente implementato con un'interfaccia REST API e Web-UI per la consultazione e il testing.

2 Organizzazione del Codice

Il progetto è suddiviso in due componenti principali, **Backend** e **Frontend**, orchestrati tramite Docker Compose insieme a un container MariaDB e uno Ollama.

2.1 Backend

Il backend è un server FastAPI che espone tutti gli endpoint REST. I moduli principali sono:

- **Parser** (`wikipedia.py`, `fifa.py`, `espn.py`, `yahoo.py`): il sistema implementa quattro parser specializzati che, nonostante seguino un workflow asincrono uniforme basato sulla libreria Crawl4AI, la logica di estrazione è stata studiata in modo diverso per ottimizzare l'estrazione del testo da pagine diversamente strutturate. Tutti i programmi seguono le stesse fasi:. Ogni parser implementa una funzione `estrai_*_chirurgico` che esegue pulizia preventiva dei tag non informativi, identificazione del container principale tramite espressioni regolari, estrazione selettiva dei tag fondamentali (`p`, `h1-h6`, `li`) e normalizzazione post-estrazione.

  E' importante far notare che due domini di questi (Yahoo e FIFA) presentano criticità legate al cookie wall europeo che impedisce a Crawl4AI di accedere alle pagine in modalità live, url che sono stati gestiti tramite errori specifici (come `(status_code=403, detail="WALL RILEVATO su FIFA. Usa l'HTML salvato nel DB.")`) reindirizzando l'utente a parsare l'url selezionato tramite i **gold_standard** salvati nel DB, oppure caricando a mano l'HTML grezzo nella sezione `gs_builder`.

  Stesso ragionamento è stato fatto per alcuni link del dominio ESPN che presentavano wall-antibot.

- **Parser Router** (`parser_router.py`): instrada ogni URL al parser corretto in base al dominio rilevato, sollevando un'eccezione per i domini non supportati.

- **HTML Checker** (`html_checker.py`): gestisce il caricamento di HTML prescaricato da file locali per le pagine dinamiche o protette da cookie wall.

- **Evaluation** (`evaluation.py`): implementa le metriche di valutazione quantitative dei parser.

- **Judge** (`judge.py`): gestisce la valutazione qualitativa tramite LLM locale via Ollama. E' stato scelto il modello **"llama3.2:3b"**.

- **Database** (`database.py`, `init_db.py`): gestisce la connessione a MariaDB, l'inizializzazione dello schema e il popolamento automatico all'avvio.

- **Server** (`server.py`): espone tutti gli endpoint REST specificati, con gestione delle eccezioni tramite `HTTPException`.

2.2 Frontend

Nel frontend, il modulo python, con templating Jinja2, interroga le API del backend. E' composto da quattro pagine: **Home** (stato del sistema e domini supportati), **Parser & Evaluation** (testing interattivo con confronto HTML/testo e metriche), **GS Builder** (gestione del Gold Standard nel database) e **Stats** (statistiche aggregate per dominio). Qualora l'URL analizzato faccia parte del dataset di riferimento, vengono visualizzate le metriche di accuratezza (Precision, Recall, F1-score) assieme allo score associato dall'LLM judge.

3 Valutazione Parser

3.1 Metriche

Nel modulo evaluation.py, il testo estratto dal parser e il testo all'interno del Gold Standard vengono normalizzati in minuscolo e suddivisi in set di parole isolate tramite `token_level_eval`: le metriche calcolate sono Precision (token corretti estratti sul totale estratto), Recall (token corretti estratti sul totale del Gold Standard) e F1-score (media armonica tra le due).

3.2 Risultati

Dominio	Precision	Recall	F1
JudgeScore			
ESPN	0.990	0.988	0.989
4.00/5			
Wikipedia	0.990	0.99	0.994
3.90/5			
FIFA	0.913	0.992	0.949
3.90/5			
Yahoo Finance	0.962	0.998	0.979
4.00/5			

Table 1: Risultati medi della valutazione per dominio + media LLM Score

4 Valutazione LLM Judge

4.1 Selezione del Modello

Tra i modelli proposti compatibili con esecuzione su CPU (gemma4:e2b, llama3.2:3b, phi4-mini, qwen3:4b, ministral-3:3b), è stato selezionato **llama3.2:3b**: la scelta del modello influenza particolarmente la velocità e la precisione di calcolo della funzione `full_gs_eval` dove, sebbene tutti i modelli restituiscono buoni valori di score per i parsing degli url registrati nel DB, non tutti riescono ad essere abbastanza veloci per non registrare timeout.

4.2 Prompt

Il prompt è strutturato per guidare il modello a produrre un output JSON con due campi obbligatori: `score` (intero da 1 a 5) e `feedback` (stringa in italiano) secondo la scala di valutazione esplicitata nel programma, con descrizioni per ogni livello. Per gestire testi molto lunghi ed evitare timeout su CPU, il testo viene troncato a 200 parole totali suddivise in tre sezioni (inizio, centro, fine) tramite la funzione `tronca_testo`, così da cercare di mantenere il controllo sulla totalità del testo estratto.

L'output del modello viene estratto tramite regex e parsed come JSON, con fallback a score neutro in caso di risposta malformata.

5 Organizzazione del Database

Il database MariaDB è organizzato in cinque tabelle collegate tramite chiavi esterne:

- **web_resources**: contiene URL, dominio, titolo e HTML grezzo di ogni risorsa. Chiave primaria: `url`.
- **gold_standard**: contiene il testo di riferimento costruito manualmente. Chiave esterna verso `web_resources`.
- **parsed_documents**: memorizza i testi parsati prodotti dai parser.
- **evaluations**: archivia i risultati delle metriche quantitative (precision, recall, F1) per ogni URL.
- **llm_judgments**: memorizza score e feedback prodotti dall'LLM judge per ogni URL.

All'avvio del sistema, lo script `init_db.py` popola automaticamente `web_resources`, `gold_standard` ed `evaluations` leggendo i file JSON presenti in `gs_data/`. Le metriche token vengono pre-calcolate usando i parser reali sull'HTML già in DB, così da rendere l'endpoint `/full_gs_eval` immediato nella risposta.